SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: **Database Management Systems**
COURSE CODE: **CSA05**

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.* (BL2, BL3)

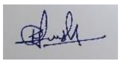
Activity Tool : Experiments (High)
Assessment Tool Weightage: 40%

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.	BL2 – Understand	**5**
2	Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.	BL3 – Apply	**5**
3	Given relation R(A,B,C,D,E) with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.	BL3 – Apply	**5**
4	Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.	BL3 – Apply	**5**
5	Verify whether R(A,B,C,D) with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.	BL3 – Apply	**5**
6	Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.	BL3 – Apply	**5**
7	Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.	BL3 – Apply	**5**

8	Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).	BL3 – Apply	**5**
9	Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.	BL2 – Understand	**5**
10	Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.	BL3 – Apply	**5**

Code of Conduct:

I K. Lokesh Chandra(192425358) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with	Organized documentation with	Basic documentation with	Poor or incomplete documentation

		diagrams, code, and results	necessary details	missing details	

STUDENT_COURSE – Functional Dependencies and Candidate Keys

Assume:

- Each student has a unique **SID**.
- Each course has a unique **CID**.
- A student can register for many courses.
- A course can have many students.
- Grade depends on both student and course.

Functional Dependencies

1. **SID → SName**
    - Student ID uniquely identifies the student's name.
2. **CID → CName**
    - Course ID uniquely identifies the course name.
3. **CID → Instructor**
    - Course ID identifies the instructor handling that course.
4. **SID, CID → Grade**
    - Grade depends on both the student and the course.

Therefore:

F = { SID → SName, CID → CName, CID → Instructor, SID,CID → Grade }

Candidate Key

Consider:

(SID, CID)+

Starting with SID and CID:

SID, CID

↓

SName

↓

CName, Instructor

↓

Grade

Therefore:

(SID, CID)+ = {SID, SName, CID, CName, Instructor, Grade}

Hence:

Candidate Key = {SID, CID}

Neither SID nor CID alone can identify every tuple.

Result

The candidate key of STUDENT_COURSE is **(SID, CID)**, and the major functional dependencies are:

SID → SName

CID → CName

CID → Instructor

SID,CID → Grade

2. Conversion of Unnormalized Relation into 1NF

The assessment asks for a step-by-step demonstration of converting an unnormalized relation into 1NF.

Answer

Consider an unnormalized STUDENT relation:

SID	StudentName	Courses
S01	Arun	DBMS, OS
S02	Ravi	Java, Python

Here, the **Courses** attribute contains multiple values.

This violates **1NF** because every attribute must contain a single atomic value.

Step 1 – Identify repeating groups

The repeating values are:

Courses = DBMS, OS

Courses = Java, Python

Step 2 – Separate repeating values

Convert each course into a separate row.

1NF Relation

SID	StudentName	Course
S01	Arun	DBMS
S01	Arun	OS
S02	Ravi	Java
S02	Ravi	Python

Now each cell contains only **one atomic value**.

Step 3 – Verify 1NF

The relation satisfies:

- No repeating groups.
- No multivalued attributes.
- Each cell contains one value.
- Each row represents one record.

Result

The unnormalized relation has been successfully converted into **First Normal Form (1NF)**.

UNF

↓

Remove repeating/multivalued attributes

↓

1NF

↓

Atomic values in every cell

3. Decomposition to 2NF

Given:
R(A,B,C,D,E)
FDs:
$A \rightarrow BC$
$BC \rightarrow D$
$D \rightarrow E$
The question asks to analyze and decompose the relation into 2NF by eliminating partial dependencies.
Step 1 – Find Candidate Key
Start with A:
$A+$
$= \{A\}$
Using:
$A \rightarrow BC$
we get:
$A+ = \{A,B,C\}$
Using:
$BC \rightarrow D$
we get:
$A+ = \{A,B,C,D\}$
Using:
$D \rightarrow E$
we get:
$A+ = \{A,B,C,D,E\}$
Therefore:
A is the candidate key.
Step 2 – Check for Partial Dependency
A partial dependency occurs when a non-prime attribute depends on **part of a composite candidate key**.
Here the candidate key is:
A
It consists of only one attribute.
Therefore, a partial dependency **cannot exist**.
Important Observation
The relation is **already in 2NF**.
However, we can optionally decompose it based on the dependencies:
R1(A,B,C)
R2(B,C,D,E)
R1
R1(A,B,C)

FD:
A → BC
Candidate key:
A
R2
R2(B,C,D,E)
FDs:
BC → D
D → E
Candidate key:
BC
There is no partial dependency on part of BC.
Result
The original relation is already in **2NF** because its candidate key is a single attribute **A**. The optional decomposition preserves the 2NF property.

4. 3NF Decomposition of EMPLOYEE

Given:
EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID)
The question specifically asks for 3NF decomposition with transitive dependencies.
Step 1 – Identify Functional Dependencies
Assume:
EmpID → EmpName
EmpID → DeptID
DeptID → DeptName
DeptID → ManagerID
Therefore:
EmpID → DeptID → DeptName
EmpID → DeptID → ManagerID
These are **transitive dependencies**.
Step 2 – Identify the Problem
The primary key is:
EmpID
But:
EmpID → DeptID
DeptID → DeptName
So:
EmpID → DeptName
indirectly.
Similarly:
EmpID → DeptID
DeptID → ManagerID

This causes redundancy.

Step 3 – Decompose into 3NF

Create:

EMPLOYEE

EmpID	EmpName	DeptID
E01	Arun	D01
E02	Ravi	D02

DEPARTMENT

DeptID	DeptName	ManagerID
D01	CSE	M01
D02	ECE	M02

Functional Dependencies

EMPLOYEE:
EmpID → EmpName, DeptID

DEPARTMENT:
DeptID → DeptName, ManagerID

3NF Structure

EMPLOYEE
EmpID (PK)
EmpName
DeptID (FK)
|
↓
DEPARTMENT
DeptID (PK)
DeptName
ManagerID

Result

The transitive dependencies are removed and the relation is decomposed into **3NF**.

5. BCNF Verification and Decomposition

Given:

R(A,B,C,D)

FDs:
AB → C
C → D
D → A

The question asks to verify BCNF and decompose if required.

Step 1 – Find Candidate Keys

AB closure
(AB)+
= {A,B}
Using:
AB → C
= {A,B,C}
Using:
C → D
= {A,B,C,D}
Therefore:
AB is a candidate key.
BC closure
(BC)+
= {B,C}
Using:
C → D
we get D.
Using:
D → A
we get A.
Therefore:
(BC)+ = {A,B,C,D}
So **BC is a candidate key**.
Similarly:
(BD)+ = {A,B,C,D}
So **BD is also a candidate key**.
Step 2 – BCNF Condition
A relation is in BCNF if:
For every non-trivial FD X → Y, X must be a superkey.
Check:
AB → C
AB is a candidate key → **satisfies BCNF**
C → D
C is **not** a superkey → **violates BCNF**
D → A
D is **not** a superkey → **violates BCNF**
Therefore
R is NOT in BCNF.
Step 3 – Decompose
Use the violating FD:
C → D
Create:
R1(C,D)
R2(A,B,C)

R1(C,D)
FD:
C → D
C is a key of R1.
Therefore R1 is in BCNF.
R2(A,B,C)
FD:
AB → C
AB is a candidate key.
Therefore R2 is in BCNF.
Final Decomposition
R1(C,D)
R2(A,B,C)
Result
The original relation violates BCNF because **C → D** and **D → A** have determinants that are not superkeys. The decomposition produces BCNF relations.

6. Lossless-Join Decomposition Using Chase Algorithm

The assessment asks for experimental verification of a lossless-join decomposition using the tabular/Chase algorithm.
Note: The question does not provide a specific relation or FD set. Therefore, the following is a standard experimental example.
Consider:
R(A,B,C)
FD:
B → A
Decomposition:
R1(A,B)
R2(B,C)
Step 1 – Construct Chase Table
Use distinguished symbols:
a, b, c

Row	A	B	C
R1(A,B)	a	b	c1
R2(B,C)	a1	b	c

Here:

- a, b, c are distinguished symbols.
- a1, c1 are new symbols.

Step 2 – Apply Functional Dependency
Given:
B → A
Both rows have:

B = b
Therefore, their A values must be made equal.
So:
a1 → a
The table becomes:

Row	A	B	C
R1	a	b	c1
R2	a	b	c

Step 3 – Check Result
The row contains:
A = a
B = b
C = c
All distinguished symbols occur in the same row.
Therefore, the decomposition is **lossless**.
Result
The Chase algorithm verifies that:
R = R1 ⋈ R2
without generating spurious tuples.
Hence the decomposition is **lossless-join**.

7. MVDs and 4NF Decomposition

Given:
TEACHER(TID, Subject, Hobby)
The question asks to identify multivalued dependencies and decompose into 4NF.
Step 1 – Understand the Situation
Suppose a teacher can teach multiple subjects and have multiple hobbies independently.
Example:

TID	Subject	Hobby
T01	DBMS	Music
T01	DBMS	Cricket
T01	OS	Music
T01	OS	Cricket

Subjects and hobbies are independent.
Step 2 – Identify MVDs
The multivalued dependencies are:
TID →→ Subject
TID →→ Hobby
This means that for one TID, multiple subjects and multiple hobbies can exist independently.
Step 3 – 4NF Violation
A relation is in 4NF if every non-trivial MVD has a determinant that is a superkey.

Here:
TID →→ Subject
but TID alone is not a superkey of the original relation.
Therefore, the relation violates **4NF**.
Step 4 – Decomposition
Create two relations:
TEACHER_SUBJECT
TEACHER_SUBJECT(TID, Subject)

TID	Subject
T01	DBMS
T01	OS

TEACHER_HOBBY
TEACHER_HOBBY(TID, Hobby)

TID	Hobby
T01	Music
T01	Cricket

Result
TEACHER(TID, Subject, Hobby)
↓
4NF Decomposition
↙ ↘
TEACHER_SUBJECT TEACHER_HOBBY
The independent multivalued facts are separated, eliminating redundancy.

8. Join Dependency and 5NF

The question asks to demonstrate redundancy caused by join dependencies and decompose the relation into 5NF.
Note: The assessment does not provide a specific relation, so the following is a standard experimental example.
Consider:
SUPPLIER_PART_PROJECT
(Supplier, Part, Project)
Suppose the business rule states that the relationship among suppliers, parts and projects can be represented by three independent relationships.
Step 1 – Identify Redundancy
Suppose:

Supplier	Part	Project
S1	P1	J1
S1	P1	J2
S1	P2	J1

Supplier	Part	Project
S1	P2	J2

The same supplier-part and supplier-project information is repeated across multiple rows. This causes redundancy.

Step 2 – Join Dependency

The relation can be represented using:

SP(Supplier, Part)

SJ(Supplier, Project)

PJ(Part, Project)

The join dependency is:

*(SP, SJ, PJ)

Meaning the original relation can be reconstructed by joining these three projections.

Step 3 – Decompose

SUPPLIER_PART

SP(Supplier, Part)

SUPPLIER_PROJECT

SJ(Supplier, Project)

PART_PROJECT

PJ(Part, Project)

Structure

```
SUPPLIER_PART_PROJECT
        ↓
      5NF
    ↙  ↓  ↘
  SP    SJ   PJ
```

Step 4 – Verify

The original relation can be reconstructed as:

SP ⋈ SJ ⋈ PJ

If the business rule guarantees the join dependency, the decomposition is lossless and removes unnecessary redundancy.

Result

The relation is decomposed into smaller relations based on the **join dependency**, achieving **Fifth Normal Form (5NF)**.

9. College Database – Anomalies and 3NF

The question asks to identify insertion, deletion and update anomalies in a poorly designed college schema and normalize it to 3NF.

Since no specific poorly designed schema is supplied in the question, the following is a representative college example.

Consider:

COLLEGE(

StudentID,

StudentName,
DeptID,
DeptName,
CourseID,
CourseName,
Instructor,
Grade
)

Functional Dependencies

StudentID → StudentName, DeptID

DeptID → DeptName

CourseID → CourseName, Instructor

StudentID, CourseID → Grade

1. Insertion Anomaly

Suppose a new department is created but no student has joined it yet.
It becomes difficult to insert the department information because the table expects a StudentID.

Problem

Department information cannot be stored independently.

2. Deletion Anomaly

If the last student belonging to a department is deleted, the department information may also be lost.

Problem

Deleting student information unintentionally deletes department information.

3. Update Anomaly

Suppose:
DeptID = D01
DeptName = Computer Science
appears in 100 student records.
If the department name changes, all 100 records must be updated.
If one record is missed, inconsistent data occurs.

3NF Normalization

Step 1 – STUDENT

STUDENT(StudentID, StudentName, DeptID)

StudentID	StudentName	DeptID
S01	Arun	D01
S02	Ravi	D02

Step 2 – DEPARTMENT

DEPARTMENT(DeptID, DeptName)

DeptID	DeptName
D01	CSE
D02	ECE

Step 3 – COURSE

COURSE(CourseID, CourseName, Instructor)

CourseID	CourseName	Instructor
C01	DBMS	Kumar
C02	OS	Ravi

Step 4 – ENROLLMENT

ENROLLMENT(StudentID, CourseID, Grade)

StudentID	CourseID	Grade
S01	C01	A
S01	C02	B

Final 3NF Structure

```
STUDENT
  |
  | DeptID
  ↓
DEPARTMENT

STUDENT
  |
  | StudentID
  ↓
ENROLLMENT
  ↑
  | CourseID
COURSE
```

Result

The database is normalized to **3NF**, reducing insertion, deletion and update anomalies and improving data integrity.

10. Attribute Closure and Candidate Keys

The question asks to compute attribute closures and determine all candidate keys.
The assessment question does not specify an FD set for Q10. Therefore, the following is a worked example using a valid FD set.
Consider:
R(A,B,C,D)
FDs:
AB → C
C → D
D → A
1. Closure of AB
Start:
(AB)+ = {A,B}
Using:
AB → C
we get C:
{A,B,C}
Using:
C → D
we get D:
{A,B,C,D}
Therefore:
(AB)+ = {A,B,C,D}
So **AB is a candidate key**.

2. Closure of BC
Start:
(BC)+ = {B,C}
Using:
C → D
we get:
{B,C,D}
Using:
D → A
we get:
{A,B,C,D}
Therefore:
(BC)+ = {A,B,C,D}
So **BC is a candidate key**.

3. Closure of BD
Start:

(BD)+ = {B,D}
Using:
D → A
we get:
{A,B,D}
Using:
AB → C
we get:
{A,B,C,D}
Therefore:
(BD)+ = {A,B,C,D}
So **BD is a candidate key**.

Candidate Keys
Therefore:
Candidate Keys:
AB
BC
BD
Closure Summary

Attribute Set	Closure	Candidate Key
AB	ABCD	Yes
BC	ABCD	Yes
BD	ABCD	Yes
A	A	No
B	B	No
C	ACD	No
D	AD	No

Result
The attribute closure method successfully identifies all minimal attribute sets that determine all attributes of R.